REPRODUCIBLE LORA INFERENCE THROUGH DETERMINISTIC SPLIT-K REDUCTION

Dhayaneshwar Balusamy
JPMorgan Chase & Co
dhayaneshwar.balusamy@chase.com

Ankur Mohan
JPMorgan Chase & Co
ankur.mohan@chase.com

arXiv:2611.10334v1 [cs.LG] 7 Nov 2026

ABSTRACT

Reproducibility is important for LoRA-adapted language models used in repeated evaluation, auditing, and decision-making. However, Split-K kernels used to introduce parallelism in low-rank LoRA multiplications can have the unintended consequence of numerical nondeterminism when partial results are combined using floating-point atomic additions. We investigate this non-determinism in vLLM across GSM8K mathematical reasoning, MBPP code generation, and Jev-style structured toxicity decisions on Civil Comments and introduce a deterministic Triton Split-K implementation. Under identical-input replay, Atomic Split-K=8 produces different FP32 shrink outputs in 97.61% of comparisons, while our Deterministic Split-K=8 shows no observed variation. Repeated atomic inference changes generated outputs in 17.30% of GSM8K and 14.96% of MBPP comparisons, sometimes altering task correctness. Structured-decision probabilities vary in 81.47% of atomic comparisons, with decision flips. Controlled tracing demonstrates how shrink-output differences propagate into the KV cache and how accumulated cache differences can change token decisions and final answers. Our deterministic Triton Split-K=8 implementation preserves parallelism, achieving 16.8-21.1% latency-based speedup over unsplit reduction with only 1.02-2.62% additional latency relative to Atomic Split-K=8.

1 INTRODUCTION

Low-Rank Adaptation (LoRA) has become a widely adopted approach for adapting large language models (LLMs) to downstream tasks without duplicating their pretrained weights (Hu et al., 2022). By introducing compact, task-specific parameters, LoRA enables efficient specialization and deployment of multiple models on shared inference infrastructure. Systems such as Punica and SLoRA demonstrate the importance of specialized GPU execution and efficient adapter serving (Chen et al., 2024; Sheng et al., 2024). While inference systems are typically optimized for throughput and latency, reproducibility is also important when model outputs must be audited, compared, or used to make decisions. Repeated inference with identical model parameters, inputs, and decoding configurations should ideally produce consistent results. However, numerical properties of GPU execution can undermine this expectation even under greedy decoding (He & Thinking Machines Lab, 2025).

We encountered this issue while evaluating LoRA-adapted guardrail models that assess generated content against safety, policy, and application-specific requirements, including toxicity, prompt-injection, and adherence to the supplied context. During evaluation, we observed inconsistent numerical outputs across nominally identical inference executions. Such variability raises concerns for guardrail applications, where small changes in prediction scores may affect threshold-based decisions and inconsistent results can complicate auditing and repeated evaluation. This motivated a systems-level investigation into whether LoRA inference could provide repeatable predictions without sacrificing the parallelism needed for efficient serving.

To study this behavior systematically, we use three publicly available datasets. We evaluate GSM8K mathematical reasoning and MBPP Python code generation to investigate numerical instability during autoregressive generation. In these tasks, a difference in an early token selection changes the context for subsequent predictions, potentially leading to a different final numerical answer or generated program. However, changes in generated outputs do not necessarily imply changes in task correctness. We extend our investigation to Jev-style structured decisions using vLLM's structured-decision interface (vLLM Project, 2026) with the `noul` (yes/no) question type, evaluating the stability of returned toxicity probabilities and binary decisions on Civil Comments. This setting allows us to examine numerical variation in prediction scores and its potential effect on decisions near the classification boundary.

Our investigation focuses on the LoRA *shrink* operation in

vLLM (Kwon et al., 2023). For a linear layer with a LoRA adapter, the output is computed as

$$Y = XW^{\mathsf{T}} + s\,(XA^{\mathsf{T}})B^{\mathsf{T}}, \tag{1}$$

where $A \in \mathbb{R}^{r \times H}$ and $B \in \mathbb{R}^{H_{\text{out}} \times r}$ are the low-rank projection matrices, r is the adapter rank, and s is the scaling factor. The shrink operation, XA^{T}, reduces the input hidden dimension to the much smaller adapter rank. To improve GPU utilization, Split-K parallelism divides the reduction dimension across multiple thread blocks. In the atomic Split-K implementation examined in vLLM v0.29.0, independently computed partial results are accumulated using floating-point atomic additions. The order of these additions depends on GPU execution scheduling, and because floating-point addition is not associative (Goldberg, 1991), different accumulation orders can produce different intermediate values despite identical inputs and weights.

The consequences of this numerical variation depend on its propagation through subsequent model computations. Differences in FP32 shrink outputs may disappear during precision conversion, while others may survive and perturb downstream activations and logits. A change in logits does not necessarily affect the selected token, but sufficiently large perturbations relative to the decision margin can change the prediction. During autoregressive generation, such a change may propagate through subsequent tokens, producing divergent responses and potentially different task outcomes.

We investigate these effects by comparing three LoRA shrink implementations: Atomic Split-K=8, unsplit Split-K=1, and our Deterministic Split-K=8 implementation. In our implementation, independent GPU thread blocks compute FP32 partial results, while integer atomic counters identify the last-finishing block, which combines the partials in a fixed order. We evaluate numerical reproducibility through identical-input kernel replays, its impact on model predictions and task correctness through repeated inference, and the performance trade-offs among the three implementations.

1.1 Our Contributions

This work makes the following contributions:

1. **Characterization of LoRA shrink nondeterminism.** We isolate numerical variability within vLLM's atomic Split-K LoRA shrink operation by replaying identical activations and adapter weights. Across prefill and decode, we quantify how unordered FP32 accumulation produces different outputs under otherwise identical execution conditions.
2. **Analysis of application-level consequences.** We systematically evaluate how numerical variability affects generated outputs and task correctness on GSM8K and MBPP, as well as toxicity probabilities and binary decisions through Jev-style structured-decision interface on Civil Comments. A detailed case study further traces numerical differences through the LoRA shrink operation and KV cache, using controlled cache replacement to reproduce a changed token decision and final answer.
3. **Deterministic parallel LoRA shrink.** We implement a deterministic Triton Split-K kernel that preserves parallelism and uses per-tile counters to enable fixed-order FP32 reduction without a separate reduction launch. Our implementation achieves lower latency than an existing deterministic Split-K kernel while eliminating observed run-to-run variability under fixed execution configurations. It delivers an average 19.2% speedup over Split-K=1 with only 1.69% latency overhead relative to nondeterministic Atomic Split-K=8.

2 Background: Split-K in LoRA Shrink

The LoRA shrink operation in Equation 1 projects an input $X \in \mathbb{R}^{M \times H}$ into a low-rank representation of shape $[M, r]$. Unlike conventional transformer projections, whose output dimensions are typically large, the shrink operation produces only r output features per token, where $r \ll H$. This makes the operation a narrow matrix multiplication with limited output parallelism (Chen et al., 2024).

For a GPU kernel that computes output tiles of size $B_M \times B_N$, the number of independent thread blocks is approximately

$$N_{\text{tiles}} = \left\lceil \frac{M}{B_M} \right\rceil \left\lceil \frac{r}{B_N} \right\rceil. \tag{2}$$

For example, with $M = 256$, $r = 16$, $B_M = 32$, and $B_N = 16$, the shrink operation exposes eight output tiles, corresponding to eight thread blocks in the unsplit computation. A conventional transformer projection with a much wider output dimension can expose hundreds or thousands of tiles for the same number of input tokens. Consequently, even during prefill, a low-rank shrink operation may not provide enough independent parallel work to utilize the GPU effectively.

The limitation can be more pronounced during autoregressive decode, where the number of active tokens may be much smaller. Thus, both single-token classification workloads, which primarily exercise prefill, and longer generation workloads, which include repeated decode steps, can benefit from additional parallelism along the reduction dimension.

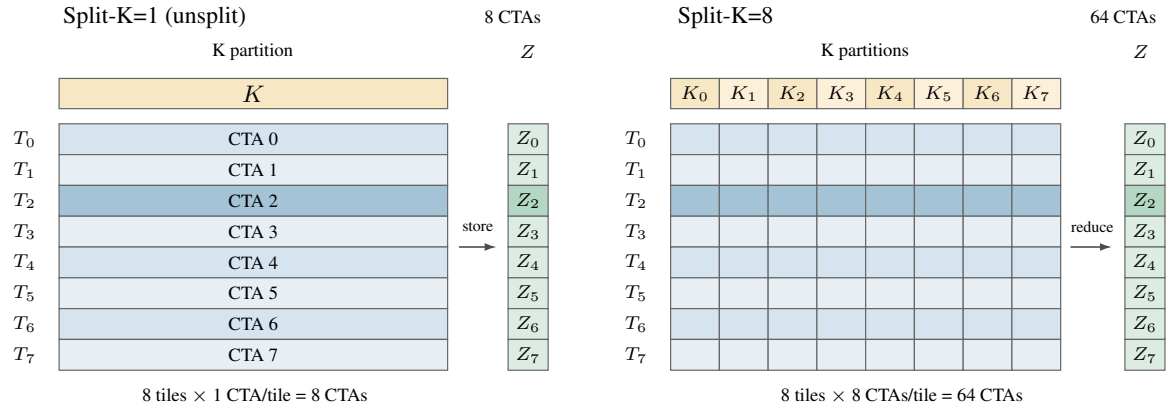

Figure 1. Thread-block parallelism in LoRA shrink. Split-K=1 assigns one CTA per output tile, whereas Split-K=8 assigns eight CTAs per tile across independent K partitions. The highlighted row illustrates one output tile.

2.1 Split-K Decomposition

Split-K increases parallelism by partitioning the reduction dimension H into S subsets $\mathcal{K}_1, \ldots, \mathcal{K}_S$. Each partition computes a partial result:

$$Z_{ij}^{(s)} = \sum_{k \in \mathcal{K}_s} X_{ik} A_{jk}, \qquad Z_{ij} = \sum_{s=1}^{S} Z_{ij}^{(s)}. \tag{3}$$

In the unsplit configuration, one thread block computes each output tile. Split-K instead assigns S independent thread blocks per output tile, each processing a different subset of the reduction dimension. This increases the number of thread blocks from N_{tiles} to SN_{tiles}. As illustrated in Figure 1, for eight output tiles, Split-K=8 increases the number of thread blocks from 8 to 64.

This additional parallelism comes at the cost of storing or combining partial results. Its benefit depends on the input dimensions, LoRA rank, reduction overhead, and GPU architecture. When the original output grid already provides sufficient parallelism, splitting the reduction dimension may provide little benefit.

To illustrate this benefit, we profile the native LoRA shrink operation on an NVIDIA RTX PRO 6000 Blackwell GPU (188 SMs). For a one-token decode operation with BF16 operands, hidden dimension 3,072, rank 16, and three fused QKV slices, Split-K=8 increases the number of thread blocks from 3 to 24 (8x). Nsight Compute reports an increase in SM activity from 1.46% to 8.06%. Separately, unprofiled CUDA-graph measurements show that complete shrink latency, including output initialization, decreases from 14.16 to 4.08 μs, a 3.47$\times$ speedup over Split-K=1.

2.2 Order Dependence and Downstream Observability

In the atomic Split-K implementation examined in this work, independent thread blocks accumulate their FP32 partial results through floating-point atomic additions. Although atomic operations ensure that concurrent updates do not overwrite one another, the order in which thread blocks accumulate their partial results can vary across executions. Because floating-point addition is not associative in general (Goldberg, 1991; Higham, 1993), different evaluation orders can produce different results:

$$\text{fl}(\text{fl}(a+b)+c) \neq \text{fl}(a+\text{fl}(b+c)). \tag{4}$$

For example, consider three partial values $a = 10^{30}$, $b = -10^{30}$, and $c = 1$. In finite-precision arithmetic, evaluating $(a+b)+c$ produces 1, whereas $a+(b+c)$ produces 0, because the addition of 1 to -10^{30} is rounded away. Thus, different accumulation orders can produce different results even when the operands are identical.

The significance of these differences depends on subsequent computation. Some FP32 differences disappear when converted to a lower precision, while others survive and may perturb downstream activations or logits. Such perturbations do not necessarily change the selected token or task outcome. We therefore distinguish numerical variation at the shrink operation from differences that propagate to model predictions.

Accumulating Split-K partial results in a fixed order removes schedule-dependent floating-point accumulation from this operation (Demmel & Nguyen, 2013). However, it does not guarantee equivalence to an unsplit reduction or reproducibility of the entire model.

3 Models, Datasets, and Adapters

The work originated from a numerical reproducibility issue observed in an internal production system, where the resulting deterministic kernel was subsequently deployed. To characterize the issue and share reproducible evidence without relying on proprietary workloads, the experiments reported here use publicly available models and datasets.

3.1 Datasets and Evaluation

We evaluate mathematical reasoning using GSM8K (Cobbe et al., 2021), which contains 7,473 training and 1,319 test problems. We reserve 5% of the training set for validation using a fixed seed, yielding 7,099 training and 374 validation examples. Preprocessing removes calculator annotations and replaces the original `####` delimiter with `Final answer:`. At evaluation, the model receives the question and an instruction to provide concise reasoning followed by a numerical answer. We extract the last `Final answer:` value and compare it with the reference using exact numerical equality; missing or unparseable answers are counted as incorrect.[1]

For code generation, we use the MBPP `full` configuration (Austin et al., 2021), containing 374 training, 90 validation, and 500 test tasks. Prompts include the task description, optional setup code, and public test assertions, but not the reference implementation. Generated programs are evaluated against the provided assertions in a network-disabled, resource-limited Docker container.[2]

For toxicity classification, we use Civil Comments (Borkan et al., 2019). Binary labels are obtained by thresholding dataset toxicity scores at 0.5. After removing empty comments and exact duplicates across partitions, fixed-seed, class-proportional sampling selects 20,000 training, 2,000 validation, and 1,000 test examples. The test set contains 920 non-toxic and 80 toxic comments. Each comment is evaluated through vLLM's `noul` interface using the question "Is this comment toxic?" The returned probability determines the predicted class, allowing us to measure probability and decision stability. [3]

3.2 Model families and adapter configurations

We evaluate Llama-3.2-3B-Instruct (Meta, 2024), Gemma-4-E4B-it (Google, 2026), and NVIDIA-Nemotron-3-Nano-4B-BF16 (NVIDIA, 2026) on GSM8K and MBPP. These models span different architectures: Llama uses a conventional dense Transformer, Gemma incorporates local and global attention, and Nemotron combines Mamba-2 and Transformer layers. This diversity allows us to examine LoRA shrink reproducibility across different model designs. For each model and task, we use separate LoRA adapters with ranks 16 and 32 and scaling factor $\alpha = 2r$. For Civil Comments classification, we additionally evaluate Qwen3-0.6B (Qwen Team, 2025) with a rank-16 LoRA adapter ($\alpha = 32$), targeting attention and feed-forward projections.

All experiments use BF16 base-model and adapter weights during inference, without quantization. Thinking mode is disabled for Gemma and Nemotron. For each model, task, and adapter configuration, we keep model weights, input prompts, chat templates, and decoding settings fixed across reduction schemes. We also verify that the intended LoRA shrink kernel is executed.

4 Execution Environment and Reduction Variants

The GSM8K, MBPP, and Civil Comments experiments use BF16 model and adapter weights, FP32 shrink accumulation, and no quantization. Inference uses tensor parallelism one and a GPU-memory-utilization target of 0.85. GSM8K and MBPP use a maximum model length and batched-token limit of 8,192, while Civil Comments uses 4,096 for both.

Prefix caching, asynchronous scheduling, and speculative decoding are disabled. We evaluate sequential inference with one active request at a time, along with fixed-batch inference at batch sizes 1, 16, 32, and 64. CUDA graphs are enabled for the main generation and timing experiments, while identical-input shrink capture uses eager execution. The GPU, software versions, adapter hashes, prompt IDs, and kernel dispatch are recorded. We disable `VLLM_BATCH_INVARIANT` to isolate the LoRA shrink reduction, since enabling it can also change other inference kernels.

The default LoRA shrink configuration in vLLM v0.29.0 selects Split-K=64 for $M < 128$ and Split-K=8 for $M \geq 128$, where M denotes the number of input token rows. With batch-invariant execution enabled, the examined shrink path instead selects Split-K=1.

We compare deterministic Split-K configurations with splits $S \in \{2, 4, 8, 16\}$. In our LoRA shrink kernel benchmarks, Split-K=8 is 48.1% faster than Split-K=2 and 12.6% faster than Split-K=4, while being only 2.0% slower than Split-K=16. We select Split-K=8 for our main experiments as it offers comparable performance to Split-K=16 with fewer partitions, while also aligning with vLLM's existing eight-way Split-K configuration.

[1] GSM8K dataset
[2] MBPP dataset
[3] Civil Comments dataset

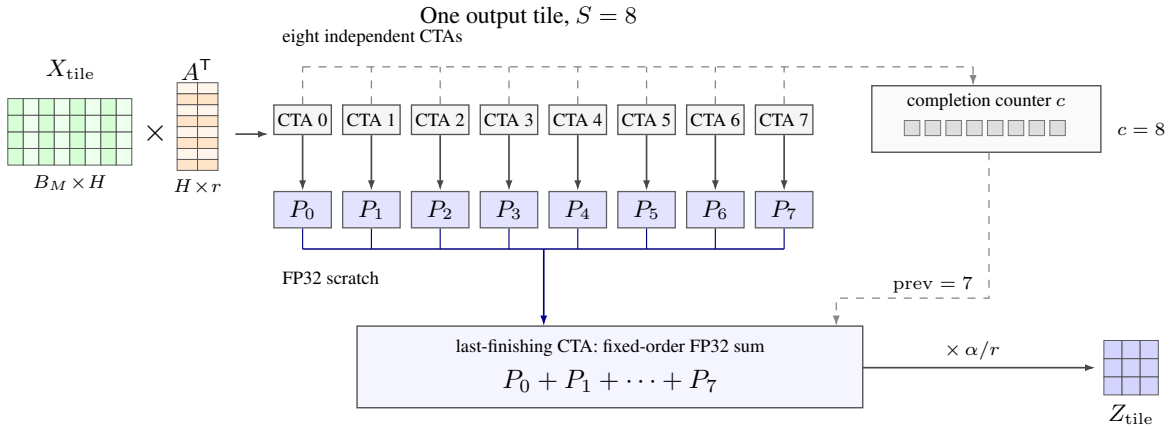

Figure 2. Deterministic Split-K=8 LoRA shrink. Eight CTAs compute FP32 partials for one output tile. The last-finishing CTA combines them in a fixed order and writes the scaled result.

4.1 Atomic Split-K=8

In the atomic Split-K=8 baseline, the reduction dimension is divided into eight partitions, each independently computing partial output tiles. For each output tile, the eight corresponding partial results are accumulated using FP32 atomic additions. Since the order of these additions depends on GPU scheduling, the accumulated values may differ across executions. We denote this configuration as `atomic_sk8`.

4.2 Unsplit Reduction (Split-K=1)

In the unsplit Split-K=1 baseline, each GPU thread block computes an output tile over the entire reduction dimension, without combining partial results from multiple blocks. This eliminates cross-partition accumulation within the LoRA shrink operation. However, its numerical results may differ from Deterministic Split-K=8 because the two implementations use different floating-point accumulation orders.

4.3 Deterministic Split-K=8

Our Triton (Tillet et al., 2019) implementation preserves eight-way Split-K parallelism while replacing floating-point atomic accumulation with fixed-order summation. The kernel supports BF16 inputs and weights with FP32 shrink outputs. An initialization kernel clears the output tensor and per-tile integer completion counters. In the subsequent kernel, eight GPU thread blocks independently compute FP32 partial results for each output tile and store them in separate locations of a scratch buffer.

Before signaling completion, a GPU-wide memory fence and a block-wide barrier ensure that the partial result is published to memory. Each block then increments the per-tile completion counter using a GPU-scoped acquire-release integer atomic operation. The block observing the previous counter value of seven is the last to signal completion (NVIDIA, 2025). It reads the eight stored partial results, accumulates them in increasing partition order using FP32 arithmetic, applies the LoRA scaling factor, and writes the final output tile. Algorithm 1 summarizes the completion-counter synchronization and fixed-order FP32 accumulation.

Algorithm 1 Deterministic Split-K=8 LoRA Shrink

```
Initialize output and completion counters to zero.
for all output tiles t and splits s ∈ {0, ..., 7} in parallel do
    p ← compute_fp32_partial(t, s)
    scratch[t, s] ← p
    gpu_memory_fence()
    block_barrier()
    v ← atomic_increment(counter[t])
    if v = 7 then
        total ← FP32(0)
        for j = 0 to 7 do
            total ← total + scratch[t, j]
        end for
        output[t] ← total × scale
        counter[t] ← 0
    end if
end for
```

Although the order in which blocks complete may vary, the floating-point accumulation always follows the same partition order. Integer atomic operations are used only to coordinate completion, not to accumulate the partial results. The implementation thus removes schedule-dependent floating-point accumulation from LoRA shrink while retaining Split-K parallelism.

We also explored an earlier deterministic implementation that computed Split-K partials in parallel and combined them using a separate kernel with a fixed pairwise reduction

tree. Although this provided a deterministic accumulation order, the additional reduction launch introduced overhead, particularly for small decode workloads. This motivated our current design, which integrates final accumulation into a partial-computation kernel.

5 Benchmark Protocol and Measurements

We compare Atomic Split-K=8, Split-K=1, and Deterministic Split-K=8 on a single NVIDIA RTX PRO 6000 Blackwell GPU. All experiments use BF16 model and adapter weights without quantization. Reproducibility and performance are measured separately to exclude diagnostic overhead from timing results.

5.1 Generation Reproducibility

For GSM8K and MBPP, we select 50 test prompts per dataset using a fixed seed. The same prompts are evaluated across three models, two LoRA ranks, and three reduction methods. Each prompt is generated ten times sequentially within the same engine using greedy decoding, with a 4,096-token output limit.

Repetitions 1-9 are compared against repetition 0, yielding 450 comparisons per configuration. We measure differences in generated token sequences, selected-token raw logits, and task correctness. Logit differences are measured only at matching-prefix positions with identical selected tokens, excluding the first divergent token and subsequent positions.

5.2 Identical-Input Shrink Replay

To isolate kernel-level variability, we capture shrink inputs, adapter weights, scaling factors, and mapping information from ten prompts per dataset under Split-K=1 in eager mode, covering prefill and the first decode step.

Each captured operation is replayed using all three reduction methods on identical inputs. After warmup, five executions are collected per method, with executions 1-4 compared against execution 0. We record the fraction of repeat comparisons with differing FP32 shrink outputs, the fraction of differing FP32 output elements, and the mean and maximum absolute differences.

5.3 Inference Performance and Batch Sensitivity

Performance is measured using 50 sequential requests with exactly 150 generated tokens per request and EOS stopping disabled, across five fresh-engine trials per configuration. We measure end-to-end latency, time to first token, time per output token, and output throughput. CUDA graphs are enabled, with initialization and warmup excluded from measurements.

For batch-size experiments, we use Llama-3.2-3B-Instruct on GSM8K with ranks 16 and 32, processing the same 64 prompts at batch sizes 1, 16, 32, and 64. Timing uses three fresh-engine trials. Separately, each fixed-batch workload is repeated ten times to measure within-batch reproducibility and differences relative to batch-size-one execution.

5.4 Structured Decision Models

Recent structured decision models provide an alternative to open-ended autoregressive generation when applications require predefined decisions and associated probabilities. Jev, introduced by TypeSafe AI, popularized this interface through three decision primitives: selecting among alternatives (`choice`), assigning an ordinal score (`score`), and estimating the probability of a binary yes/no decision (`noul`) (Almeida, 2026; Li, 2026). This approach is particularly relevant to applications such as routing, policy enforcement, and content moderation, where structured decisions are more useful than generated text.

vLLM has recently incorporated support for structured decisions (vLLM Project, 2026), including through compatible generative models. For `noul` questions, the interface returns a probability normalized over the `yes` and `no` token scores. This makes it suitable for evaluating the stability of binary decisions in LoRA-based guardrail applications.

We evaluate vLLM's structured-decision interface using Qwen3-0.6B with a rank-16 LoRA adapter ($\alpha = 32$) fine-tuned on Civil Comments. Binary toxicity labels are derived using a threshold of 0.5 on the dataset's toxicity scores. After removing empty comments and exact duplicates, we select 20,000 training, 2,000 validation, and 1,000 test examples through fixed-seed, class-proportional sampling. The adapter is trained for three epochs, selecting the checkpoint with the lowest validation loss.

Each request asks the `noul` question, "Is this comment toxic?" The interface computes a toxicity probability by normalizing the next-token log-probabilities of `yes` and `no`. A probability of at least 0.5 is classified as toxic.

For each reduction scheme, we evaluate the same 1,000 comments sequentially ten times within one engine, with prefix caching disabled. Comparing repetitions 1-9 against the first yields 9,000 comparisons per scheme. We measure probability variation using the fraction of changed probabilities and their mean and maximum absolute differences. Decision stability is evaluated through label flips, correctness transitions, and classification accuracy.

6 Results and Discussion

6.1 Numerical Variability in LoRA Shrink

We examine whether numerical variation originates within the LoRA shrink operation itself. Using identical captured activations and adapter weights, we compare the repeatability of Atomic Split-K=8, Split-K=1, and Deterministic Split-K=8 across all three models, both datasets, and LoRA ranks 16 and 32.

Across 27,440 captured shrink operations covering prefill and the first decode step, each implementation is executed five times, yielding 109,760 repeat comparisons per implementation. Atomic Split-K=8 produces different FP32 outputs in 107,136 comparisons (97.61%). In contrast, Split-K=1 and Deterministic Split-K=8 exhibit no observed differences across the same captured operations. This distinction is consistent with prior work showing that fixed-shape GPU computations can be repeatable when their reduction order is stable (Gond et al., 2026). Our results identify unordered accumulation in LoRA shrink as a source of run-to-run variation under the tested execution conditions.

The variability occurs in both inference phases. For Atomic Split-K=8, 99.996% of prefill comparisons and 95.22% of first-decode comparisons contain at least one changed output element (Figure 3). Across both phases, 22.31% of compared FP32 elements differ, although the mean absolute difference is only 1.08×10^{-8}.

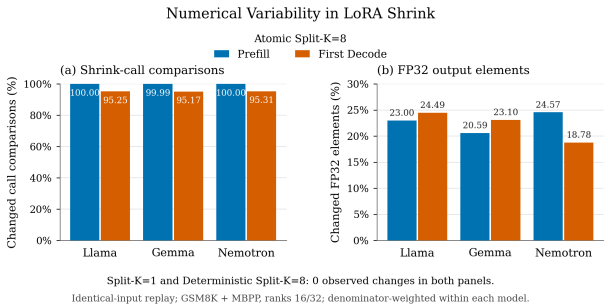

Figure 3. Run-to-run numerical variability in the LoRA shrink operation during prefill and first decode. Atomic Split-K=8 exhibits numerical variation under identical inputs, while Split-K=1 and Deterministic Split-K=8 show no observed differences.

Fixed-order accumulation eliminates the observed run-to-run variation within the tested shrink operations without removing Split-K parallelism. To assess reproducibility across GPU architectures, we additionally evaluated the implementation on NVIDIA H100 and B200 GPUs under the same experimental configuration. Neither GPU exhibited numerical variation across repeated identical-input executions, consistent with the findings obtained on the primary NVIDIA RTX PRO 6000 Blackwell platform.

6.2 Tracing LoRA Numerical Variability to Token Decisions

To trace how LoRA shrink variability propagates through autoregressive inference, we use Llama-3.2-3B-Instruct with a rank-16 GSM8K adapter. Two Atomic Split-K=8 executions of the same prompt share their first 35 generated tokens but select different tokens at position 36. Both executions terminate naturally, producing final answers of 7 (correct) and 8 (incorrect). Repeated Split-K=1 executions produce identical captured states and outputs.

To identify the origin of this divergence, we trace numerical differences from prefill through the subsequent decode steps. The first observed numerical difference originating from the atomic LoRA shrink reduction that propagates beyond the LoRA projection occurs in the layer-2 MLP up-projection, at prompt position 47. Two FP32 shrink values, 0.010345458984375 and 0.010345457121729851, differ by only 1.86×10^{-9}. The first value lies exactly at a BF16 rounding midpoint, while the second lies slightly below it. Consequently, they convert to different BF16 values (Srivastava et al., 2024), separated by 6.10×10^{-5}.

This differing BF16 element affects the subsequent MLP computation and the input to layer 3. The resulting layer-3 KV cache contains 108 differing key elements and 176 differing value elements at prompt position 47. The original differing KV-cache entries persist across the subsequent 35 token generation steps. Figure 4 illustrates the numerical differences observed during prefill and at the divergent decode step.

To verify how shrink-output differences propagate, we inject two observed shrink outputs into otherwise identical executions, using Split-K=1 for all remaining operations. This reproduces the corresponding layer-3 K/V differences at the affected prompt position, including 284 differing cache elements. However, the isolated shrink perturbation does not reproduce the later token flip.

A second controlled experiment tests whether the accumulated KV-cache differences are sufficient to change the token decision. Immediately before generating token 36, we substitute the incoming KV cache from either atomic execution and continue with Split-K=1. Both conditions are repeatable across two trials. With the first cache, the logits of `There` and `Each` are both 21.75, and the model selects `There`, producing the correct answer 7. With the second cache, the logit of `There` decreases to 21.625 while `Each` remains at 21.75. The model therefore selects `Each` and produces the incorrect answer 8.

The different tokens are followed by distinct reasoning trajectories. In the correct execution, the model subtracts the three dropped marshmallows before dividing the remainder between the two children:

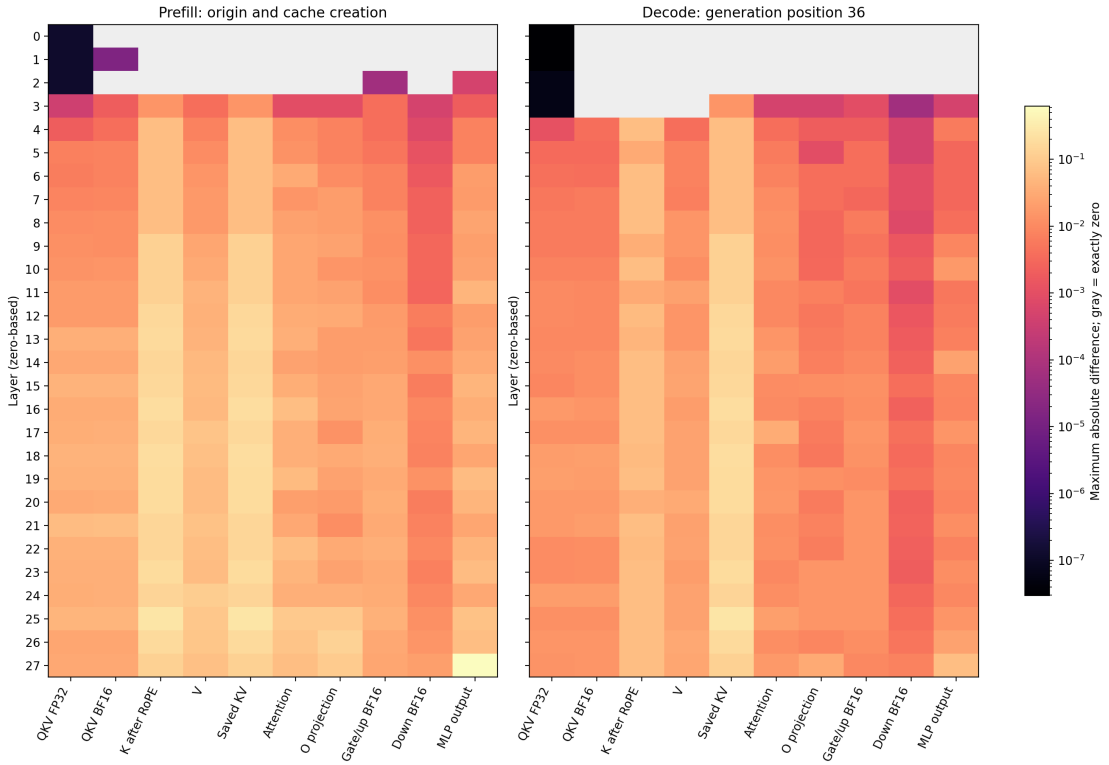

Figure 4. Numerical differences between two Atomic Split-K=8 executions during prefill and decode at generated token 36. Colors indicate maximum absolute tensor differences on a logarithmic scale; gray denotes exact agreement. The first propagating difference appears in layer 2, with cached K/V differences from layer 3 onward. Subsequent differences reflect both accumulated state variation and numerical variation during later operations.

> **Execution A (correct):**
>
> There are 35 - 18 = 17 marshmallows left in the bag.
>
> There are 17 - 3 = 14 marshmallows left in the bag.
>
> Each kid can make 14/2 = 7 S'mores.
>
> **Final answer: 7**

In the incorrect execution, the model omits the subtraction for the dropped marshmallows:

> **Execution B (incorrect):**
>
> Each kid can have 17/2 = 8.5 S'mores.
>
> Since you can't have half a S'more, each kid can have 8 S'mores.
>
> **Final answer: 8**

Although the cache intervention reproduces the token flip, it does not establish whether an individual LoRA shrink perturbation is sufficient to cause it. We therefore perform source-level interventions on ten freshly generated divergent GSM8K execution pairs using Llama-3.2-3B-Instruct with a rank-16 LoRA adapter.

We first reconstruct each atomic execution by replaying its captured shrink outputs under otherwise Split-K=1 execution. The resulting KV-cache states and target-step vocabulary logits match the original executions exactly through the first token divergence in all ten pairs. We then identify 26 shrink operations with identical incoming operands but differing FP32 outputs that survive BF16 conversion. Holding the preceding execution history fixed, we substitute individual shrink outputs with their observed alternatives and recompute all subsequent operations using Split-K=1. These interventions reproduce the corresponding atomic token choices in two of the ten prompts.

For GSM8K prompt 813, we identify a single influential FP32 coordinate in the layer-8 QKV shrink operation during prefill. Its observed values, -0.3857421278953552 and -0.3857422471046448, differ by only 1.19×10^{-7} but straddle the BF16 rounding midpoint -0.3857421875, yielding -0.384765625 and -0.38671875 after conversion. Changing only this coordinate alters 32 BF16 elements of the value projection, which are stored in the V cache, while the source-layer K cache remains unchanged. Under an identical preceding execution history and Split-K=1 for all subsequent operations, this perturbation changes the

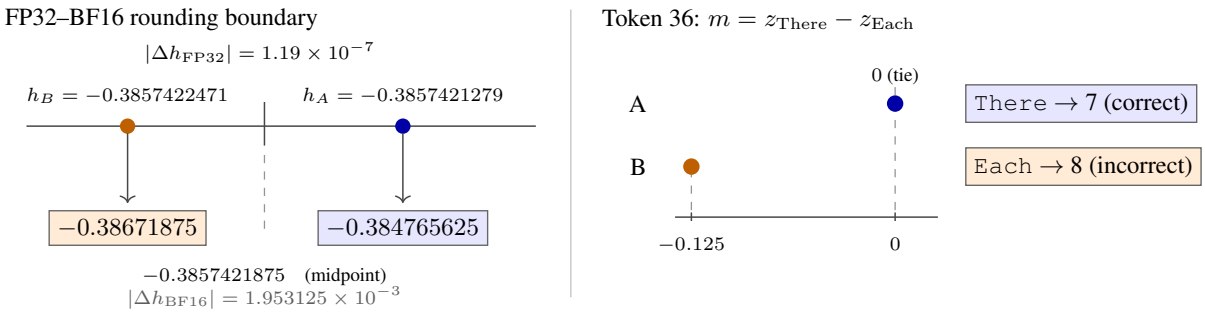

Figure 5. Single-coordinate causal intervention for GSM8K prompt 813. An observed FP32 difference crosses a BF16 rounding boundary and changes the token decision and final answer under controlled Split-K=1 continuation. FP32 positions are schematic.

selected token at generation position 36 from `There` to `Each`. Without forcing subsequent tokens, the final answer changes from 7 (correct) to 8 (incorrect), with both continuations reproducing exactly across repeated trials. Figure 5 illustrates the BF16 rounding boundary and the resulting change in token selection.

These interventions establish that an individual LoRA shrink perturbation can be sufficient to alter a downstream token decision and final task outcome. Among the ten selected divergent pairs, we identify such an influential local source in two cases, with one changing final-answer correctness. Because the pairs were selected for observed divergence, these results demonstrate causal sufficiency in specific executions rather than the frequency of such effects across the benchmark.

6.3 End-to-End Generation and Task Stability

End-to-end generation experiments assess whether numerical variability affects generated token sequences and task correctness. Prior work has shown that numerical differences during LLM inference can lead to divergent generations and changes in task outcomes (Yuan et al., 2025). For each model, dataset, and LoRA rank, we perform ten repeated greedy generations on the same 50 test prompts. Each subsequent generation is compared with the first, yielding 2,700 comparisons per dataset and reduction method across the six model-rank configurations.

Atomic Split-K=8 produces different token sequences in 467 of 2,700 comparisons (17.30%) on GSM8K and 404 of 2,700 comparisons (14.96%) on MBPP. These differences occur across all six model-rank configurations for both datasets. In contrast, Split-K=1 and Deterministic Split-K=8 show no observed output differences across the corresponding repeated executions. The atomic implementation also exhibits differences in selected-token logits at 14.57% of comparable matching-prefix positions on GSM8K and 18.30% on MBPP.

Not all output differences affect task correctness. On GSM8K, Atomic Split-K=8 produces 151 comparisons with different extracted final answers, of which 84 change correctness. These comprise 45 transitions from correct to incorrect and 39 in the opposite direction. On MBPP, 53 comparisons change whether the generated program passes the public tests provided in the prompt.

The relationship between output variability and correctness is not uniform across configurations. For example, Llama on GSM8K at rank 16 exhibits 105 changed-output comparisons and 18 correctness flips, whereas Nemotron at the same rank exhibits 77 changed outputs but 21 correctness flips. On MBPP, Nemotron at rank 16 produces 36 changed-output comparisons without any change in test outcomes. Thus, the frequency of output variation alone does not determine its effect on task correctness.

Table 1. Output and correctness changes under repeated greedy inference. Each reduction method has 2,700 comparisons per dataset, using repetitions 1-9 against repetition 0.

Dataset	Reduction	Output changes	Correctness flips
GSM8K	Atomic SK8	467 (17.30%)	84
	SK1	0	0
	Det. SK8	0	0
MBPP	Atomic SK8	404 (14.96%)	53
	SK1	0	0
	Det. SK8	0	0

The deterministic implementations eliminate the observed within-configuration output variability but do not necessarily produce identical answers across reduction methods or improve task accuracy. Their benefit in these experiments is repeatability, rather than a systematic improvement in correctness.

6.4 Jev-Style Structured Decision Stability

Atomic Split-K=8 produces different returned probabilities in 7,332 of 9,000 comparisons (81.47%). The mean abso-

lute probability difference is 0.00143, with a maximum of 0.05672. Despite this frequent probability variation, only seven comparisons (0.078%) change the binary classification, affecting two distinct comments. Split-K=1 and Deterministic Split-K=8 exhibit no observed probability or decision changes across the same repeated evaluations.

Table 2. Probability and decision stability on Civil Comments. Each reduction method has 9,000 comparisons across 1,000 comments.

Metric	Atomic SK8	SK1	Det. SK8
Probability changes (%)	81.47	0	0
Mean absolute drift	0.00143	0	0
Maximum absolute drift	0.05672	0	0
Decision flips	7	0	0
Affected comments	2	0	0
Accuracy (%)	94.25	94.20	94.30

All seven decision flips occur in two comments whose initial probability is exactly 0.5. In the flipped repetitions, the probability decreases to approximately 0.46879, changing the predicted class under the $p \geq 0.5$ threshold. Thus, frequent numerical variation in returned probabilities does not necessarily change the binary decision. The similar classification accuracies across reduction methods provide no evidence of a systematic accuracy advantage.

6.5 Repeatability Under Different Batch Sizes

Batch-size experiments test whether repeatability persists when multiple requests are processed together. Using Llama-3.2-3B-Instruct with GSM8K adapters of ranks 16 and 32, we evaluate batch sizes of 1, 16, 32, and 64. For each configuration, the same 64 prompts are processed in a fixed order, with 150 tokens generated per request. We repeat each fixed-batch workload ten times and compare repetitions 1-9 against the first.

Atomic Split-K=8 exhibits output variation at every tested batch size. Across both adapter ranks, the fraction of changed outputs ranges from 36.55% at batch size 1 to 39.93% at batch size 32. In contrast, Split-K=1 and Deterministic Split-K=8 produce identical token sequences across all repetitions at each fixed batch size.

Table 3. Output differences under fixed and changing batch sizes, aggregated across LoRA ranks 16 and 32.

Reduction	Fixed batch	Cross-batch
Atomic SK8	1748/4608 (37.93%)	168/384 (43.75%)
SK1	0/4608 (0%)	152/384 (39.58%)
Det. SK8	0/4608 (0%)	150/384 (39.06%)

We further compare the first execution at batch sizes 16, 32, and 64 against the first execution at batch size 1 for the same prompts. Despite their repeatability within each fixed batch size, Split-K=1 and Deterministic Split-K=8 exhibit output differences in 39.58% and 39.06% of these cross-batch comparisons, respectively. Atomic Split-K=8 differs in 43.75%, although this comparison also includes its inherent run-to-run variability.

Repeatability within a fixed batch size does not imply batch invariance. Eliminating schedule-dependent accumulation in the LoRA shrink operation is sufficient to obtain repeatable outputs under the tested fixed-batch conditions, but does not guarantee identical outputs across different batch sizes. Changes in batch size may alter numerical execution elsewhere in the inference stack, beyond the reduction controlled by our kernel.

6.6 Performance Cost of Deterministic Reduction

We evaluate the performance cost of deterministic LoRA shrink reduction by comparing Atomic Split-K=8, Split-K=1, and Deterministic Split-K=8 under identical inference workloads. For each model, dataset, and LoRA rank, we measure 50 sequential requests generating exactly 150 tokens each, across five independent engine trials. All measurements use CUDA graph-enabled inference, with model initialization and warmup excluded.

Across the 12 model-dataset-rank configurations, Deterministic Split-K=8 achieves a 16.8-21.1% latency-based speedup over Split-K=1, averaging 19.2%. Compared with Atomic Split-K=8, the deterministic implementation incurs 1.02-2.62% higher end-to-end latency, averaging 1.69%. Table 4 summarizes these results across models and adapter ranks.

Table 4. End-to-end latency comparison of Deterministic Split-K=8 against SK1 and Atomic Split-K=8, averaged across GSM8K and MBPP.

Model	Rank	Speedup vs. SK1 (%)	Latency overhead vs. Atomic (%)
Llama	16	21.0	1.62
	32	19.5	1.50
Gemma	16	20.6	2.49
	32	18.9	2.17
Nemotron	16	18.3	1.13
	32	16.8	1.20
Average	–	19.2	1.69

Fixed-batch performance is measured using Llama-3.2-3B-Instruct on GSM8K at batch sizes of 1, 16, 32, and 64. Each configuration processes the same 64 prompts across three independent engine trials. Deterministic Split-K=8 consistently achieves higher output throughput than Split-K=1 across both adapter ranks and all tested batch sizes, as

shown in Figure 6.

At batch size 64, Deterministic Split-K=8 achieves 6,079 and 6,060 generated tokens per second for ranks 16 and 32, respectively, compared with 5,274 and 5,257 tokens per second for Split-K=1. This represents approximately 15.3% higher throughput for both ranks. Relative to Atomic Split-K=8, the deterministic implementation incurs only 1.5-1.7% lower throughput at this batch size.

The throughput advantage over Split-K=1 varies with batch size, reaching approximately 20-21% at batch size 1 and remaining above 15% at batch size 64. Thus, deterministic reduction preserves a substantial performance advantage over unsplit execution even as batching increases.

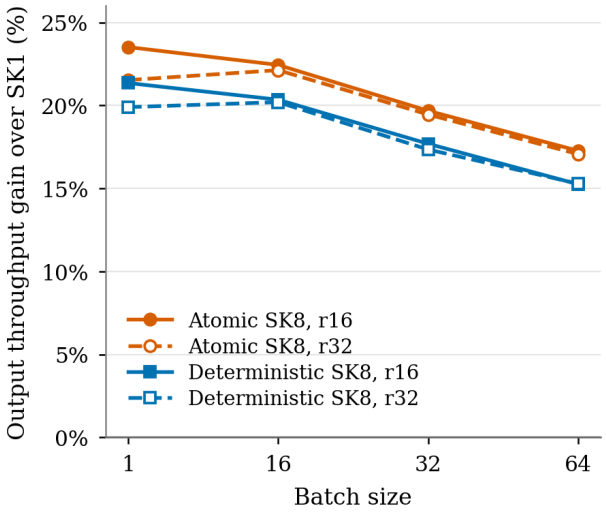

Figure 6. Output-throughput improvement of Atomic Split-K=8 and Deterministic Split-K=8 relative to Split-K=1 across batch sizes 1, 16, 32, and 64 using Llama on GSM8K.

From an engineering perspective, deterministic Split-K reduction provides a targeted alternative to disabling Split-K parallelism. The implementation achieves repeatable inference under the tested fixed configurations while retaining much of the performance benefit of parallel reduction. However, the observed cross-batch differences show that kernel-level determinism does not guarantee system-wide reproducibility. Evaluating numerical stability therefore requires testing both the LoRA shrink kernel in isolation and the complete inference pipeline under the intended batching conditions.

7 Limitations

Our deterministic Split-K implementation addresses nondeterminism from unordered floating-point accumulation in LoRA shrink but does not guarantee reproducibility across inference configurations. Although Split-K=1 and Deterministic Split-K=8 produce repeatable outputs at fixed batch sizes, changing the batch size can still alter outputs by changing execution shapes and numerical behavior in other operations. Fixed-size or bucketed batching and controlled prefix-cache reuse may help, but their effectiveness and performance trade-offs under varying workloads remain to be studied.

8 Conclusion

We demonstrate that atomic LoRA Split-K reduction introduces numerical variability that can affect model outputs and correctness. Atomic Split-K=8 changes generated outputs in 17.30% of GSM8K and 14.96% of MBPP comparisons. On Civil Comments, 81.47% of comparisons exhibit probability variation, including seven binary decision flips.

Motivated by a numerical reproducibility problem in an internal LoRA inference system, we developed and deployed a deterministic Split-K=8 kernel that preserves parallelism while combining FP32 partials in a fixed order. It eliminates the observed within-configuration variability, achieving an average 19.2% speedup over Split-K=1 with 1.69% latency overhead relative to Atomic Split-K=8. The results demonstrate that targeted kernel changes can improve inference repeatability without requiring unsplit reduction.

References

Almeida, D. Introducing system one models & jev. TypeSafe AI Blog, September 2026. https://typesafe.ai/blog/introducing-system-one-models-and-jev. Accessed October 2026.

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., and Sutton, C. Program synthesis with large language models, 2021. URL https://arxiv.org/abs/2108.07732.

Borkan, D., Dixon, L., Sorensen, J., Thain, N., and Vasserman, L. Nuanced metrics for measuring unintended bias with real data for text classification. In *Companion proceedings of the 2019 world wide web conference*, pp. 491–500, 2019.

Chen, L., Ye, Z., Wu, Y., Zhuo, D., Ceze, L., and Krishnamurthy, A. Punica: Multi-tenant lora serving. *Proceedings of Machine Learning and Systems*, 6:1–13, 2024.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve

math word problems, 2021. URL https://arxiv.org/abs/2110.14168.

Demmel, J. and Nguyen, H. D. Fast reproducible floating-point summation. In *2013 IEEE 21st Symposium on Computer Arithmetic*, pp. 163–172. IEEE, 2013.

Goldberg, D. What every computer scientist should know about floating-point arithmetic. *ACM computing surveys (CSUR)*, 23(1):5–48, 1991.

Gond, R., Kamath, A. K., Ramjee, R., and Panwar, A. Llm-42: Enabling determinism in llm inference with verified speculation. In *Proceedings of the ACM SIGOPS 32nd Symposium on Operating Systems Principles*, pp. 916–931, 2026.

Google. Gemma 4 E4B IT model card. Hugging Face, 2026. URL https://huggingface.co/google/gemma-4-E4B-it.

He, H. and Thinking Machines Lab. Defeating nondeterminism in LLM inference. *Thinking Machines Lab: Connectionism*, 2025. doi: 10.64434/tml.20250910. https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/.

Higham, N. J. The accuracy of floating point summation. *SIAM Journal on Scientific Computing*, 14(4):783–799, 1993.

Hu, E. J., yelong shen, Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=nZeVKeeFYf9.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pp. 611–626, 2023.

Li, J. Fast models, slow evidence: A paired and self-audited evaluation of system-1 decision models for llm agent harnesses. *arXiv preprint arXiv:2610.02267*, 2026.

Meta. Llama 3.2 3B Instruct model card. Hugging Face, 2024. URL https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct.

NVIDIA. CUDA C++ Programming Guide. CUDA Toolkit 12.9.1 Documentation, 2025. URL https://docs.nvidia.com/cuda/archive/12.9.1/cuda-c-programming-guide/index.html. Section 10.5, Memory Fence Functions. Accessed October 2026.

NVIDIA. NVIDIA-Nemotron-3-Nano-4B-BF16 model card. Hugging Face, 2026. URL https://huggingface.co/nvidia/NVIDIA-Nemotron-3-Nano-4B-BF16.

Qwen Team. Qwen3-0.6B model card. Hugging Face, 2025. URL https://huggingface.co/Qwen/Qwen3-0.6B.

Sheng, Y., Cao, S., Li, D., Hooper, C., Lee, N., Yang, S., Chou, C., Zhu, B., Zheng, L., Keutzer, K., Gonzalez, J. E., and Stoica, I. SLoRA: Scalable serving of thousands of LoRA adapters. In *Proceedings of Machine Learning and Systems*, volume 6, pp. 296–311, 2024.

Srivastava, M., Arora, S., and Boneh, D. Optimistic verifiable training by controlling hardware nondeterminism. *Advances in Neural Information Processing Systems*, 37: 95639–95661, 2024.

Tillet, P., Kung, H.-T., and Cox, D. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pp. 10–19, 2019.

vLLM Project. Structured decisions models. vLLM Documentation, 2026. https://docs.vllm.ai/en/latest/models/structured_decisions_models/. Accessed October 2026.

Yuan, J., Li, H., Ding, X., Xie, W., Li, Y.-J., Zhao, W., Wan, K., Shi, J., Hu, X., and Liu, Z. Understanding and mitigating numerical sources of nondeterminism in LLM inference. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=Q3qAsZAEZw.

AI USAGE DISCLOSURE

GPT-5.6 Sol was used to assist with code generation, code review, LaTeX formatting, and grammar revisions. Algorithm design, experimental planning, execution, data analysis, interpretation, and scientific conclusions were carried out by the authors.